

Answers

① The importance of requirement analysis:-

Definition - Requirement analysis is defined as a disciplined, systematic process of identifying, documenting, analyzing, validating & managing the requirements of a software system.

It ensures that the final system is aligned with the needs of Stakeholders and constraints of the business environment.

Importance :-

Requirement Analysis is considered as the foundation of successful software developments. Studies shows that 60-70% of project failure occurs due to poor/incomplete requirements.

i) Prevents scope creep: Clear requirements avoid uncontrolled changes during developments.

ii) Reduce cost and Rework: Fixing defects after deployment is 100x costlier than fixing in requirement phase.

iii) Supports Accurate Estimations: Planning, budgeting scheduling and resource allocations depends on → correct requirements.

iv Improves Stakeholder Satisfaction: Requirements represents the voice of users $\rightarrow$ better product acceptance.

v Ensure traceability: Each feature can be traced to its original need.

Requirement Engineering process:-

1 $\rightarrow$ Feasibility study: It checks whether a project is practical and worth implementing.

It is of 5 types:-

- Technical - Availability of software, hardware, tech and team skill.
- Operational - System usability, acceptability, ease of operation/maintenance.
- Economic - Cost benefits analysis.
- Legal - Compliance with laws, standards, IP-rights, contracts.
- Schedule - Can the project be completed within time constraints.

2 $\rightarrow$ Requirement Elicitation: It is defined as the process of gathering requirements from stakeholders.

- Sources - Users, customers, manuals, standards, etc.
- Techniques - Interview, surveys, observation, focus group, etc.

3 → Requirement Specification:

It is the formal documentation in a clear, complete, structured manner (Software requirement specification).

- Includes -

- Functional req. - What the system should do.
- Non-functional req. - Performance, usability, security, reliability.
- Constraints - Technical or business limitations
- Acceptance criteria - Conditions for system acceptance.

- Models used - DFD (Level-0/Level-1), ER diagrams, FDD etc).

4 → Requirement Verification & Validation:-

- Verification - Are we building the product right?
 - Ensure requirements are consistent, complete, correct and testable.

↳ Methods - Reviews, walkthroughs, buddy checks, etc.

- Validation : The requirements meets the need and expectations of stakeholders or not.

↳ Methods - Prototyping, user review, system testing etc.

5 → Requirement Management - Ongoing process of controlling and tracking requirements throughout project.

- Goal - Ensures requirement remain clear, updated, and aligned with project goals.
- Tools used - Use case, use stories, workshops, mind mapping etc.

Advantages / Disadvantages of Requirement analysis :-

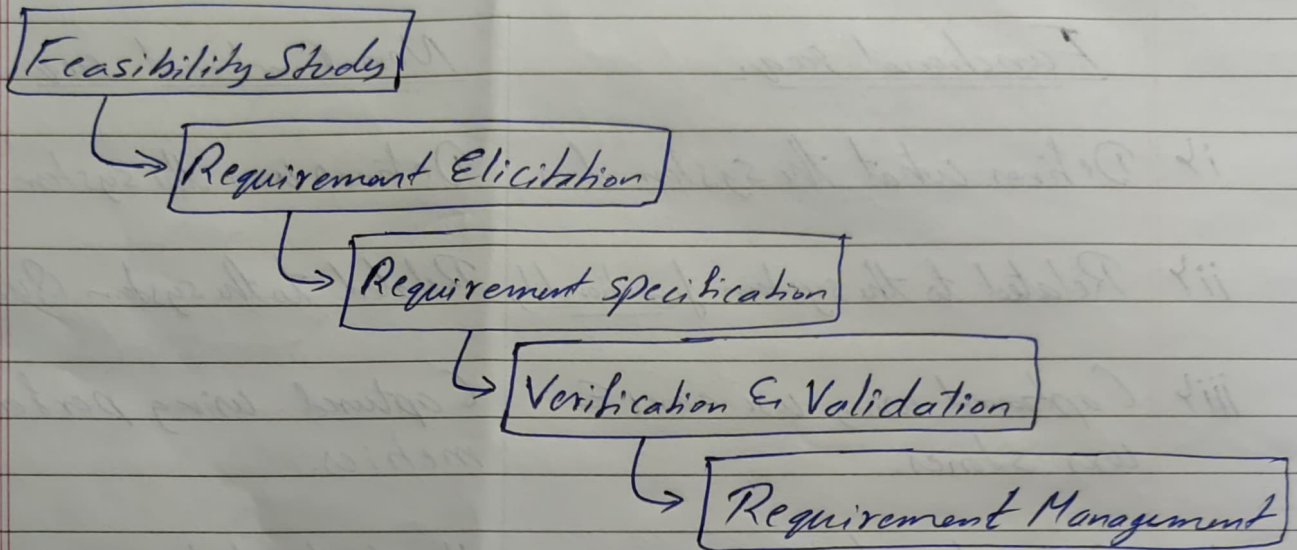
<u>Advantages</u>	<u>Dis-Advantages</u>
① Meets stakeholder needs	Time consuming, expensive.
② Save time & cost	Hard to collect all needs.
③ Ensure quality delivery.	Stake holder conflicts.

② Difference between Functional / Non-Functional req.

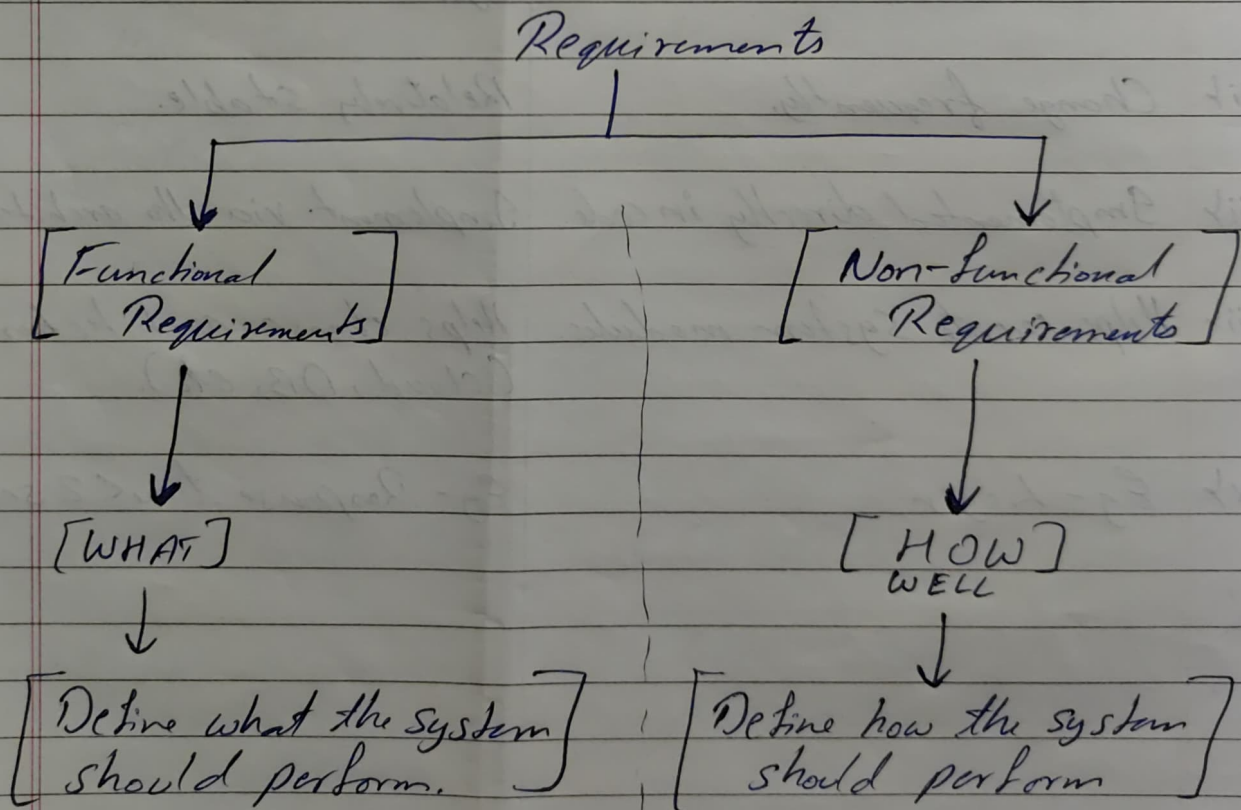
<u>Functional req.</u>	<u>Non-Functional req.</u>
i) Defines what the system does.	Defines how the system perform.
ii) Related to the system <u>functionality</u> .	Related to the system <u>Quality</u>
iii) Captured using user cases & user stories.	Captured using performance metrics.
iv) Easy to test.	Hard to test.
v) User - focused	System / Environment focused.
vi) Change frequently.	Relatively stable.
vii) Implemented directly in code.	Implement via the architecture.
viii) Helps create system modules	Helps choose architecture (cloud, DB, etc).
ix) Eg - Login	Eg - Response time < 2 sec.

Flip →

Requirement Engineering Diagram :-



FREN-FR diagram



③ Stakeholder Analysis

↳ Def. - Stakeholder analysis identifies all individuals, groups, or organisations affected by system and analyses their expectations and influence.

→ Requirement Gathering:-

It is a crucial phase in the software development life cycle and project management.

It involves collecting, documenting, managing the requirement that defines features and functionalities of system applications.

→ The main subproces includes:-

- 1) Stake holder identification.
- 2) Stake holder Analysis.
- 3) Problem definition.
- 4) Requirement Extraction.
- 5) Requirement documentation.
- 6) Validation and verification.

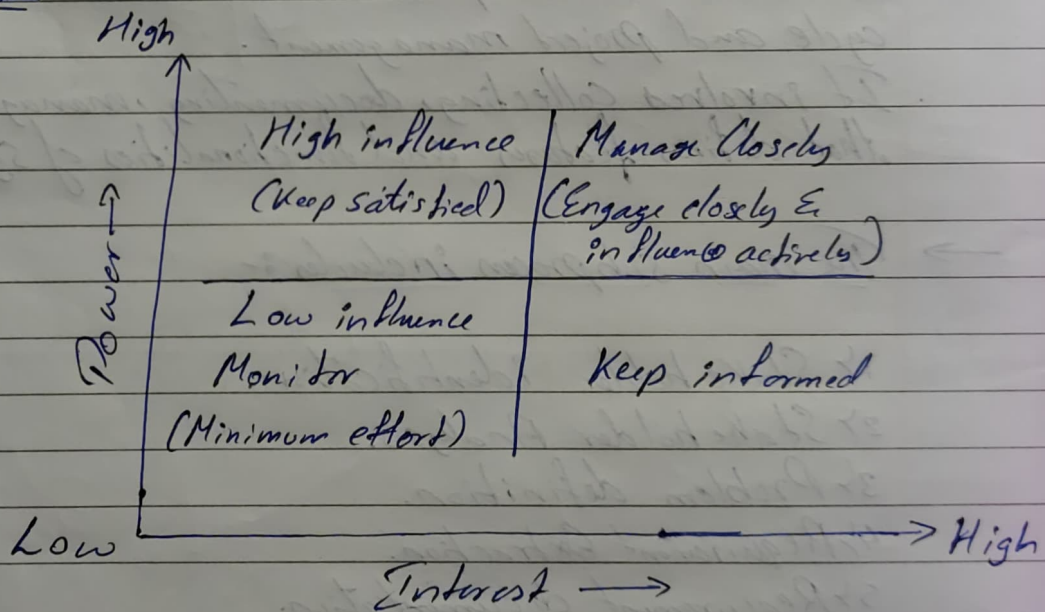
→ Why Stakeholder Analysis is Essential:-

- 1) Ensure requirement completeness.
- 2) Resolves conflicting requirements.
- 3) Identifies hidden needs.
- 4) Prioritizes features based on stakeholder importance.
- 5) Reduce project risks and misunderstanding.

→ Typical Stakeholders :-

1. End users
2. Customer / Clients
3. Management
4. Regulators
5. Software development, Team, etc.

→ Diagram



Eg - Online food delivery App :-

- Customer: Want easy ordering & Secure payment.
- Delivery Partners: Want clear delivery info.
- Restaurants: Want order management
- Admin: Wants reports & control panel.

Result → Helps identify who needs what in the system.

④ SDLC Models :- (Software Development Lifecycle)

↳ Software Modeling - It is the process of creating abstract representation of a software system. These model serves as blueprints that guide developers, designers, stakeholders through the system's structure, behavior, functionality.

↳ 5 Main types :-

- 1) Waterfall model
- 2) Agile model
- 3) Spiral model
- 4) Prototyping model
- 5) V-model.

1) Waterfall model :- It is the most famous, good version of SDLC. It is a linear and sequential model. i.e., The development phase cannot begin until the previous phase is completed.

We cannot overlap phases in the Waterfall model.

ex:- Library management system.

Requirements → Design → Coding → Testing → Deployment

Advantage

→ Easy & simple to understand.

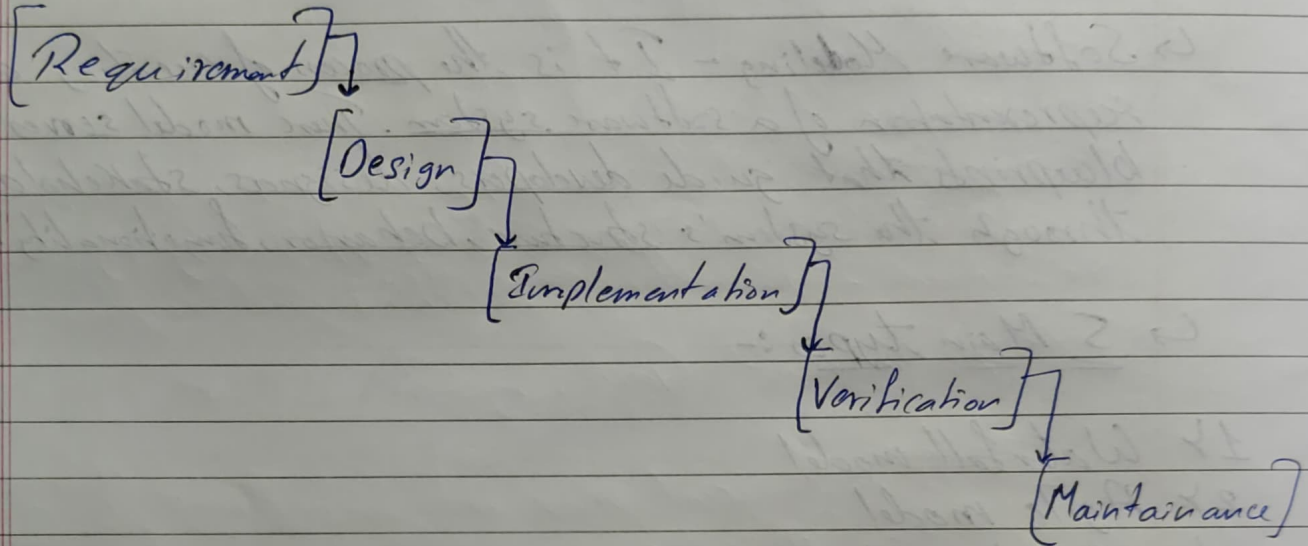
→ Clear documentation.

Disadvantage

→ No feedback loops.

→ Late testing.

→ Diagram



2. Agile Model :- It is a combination of iterative and incremental models. Focus is given to process adaptability and customer satisfaction.

It was mainly created to make changes in the middle of software development so that the software project can be completed quickly.

→ Eg:- E-commerce website -

↳ Sprint 1 → Login & Signup

↳ Sprint 2 → Product listing

↳ Sprint 3 → Cart & checkout

→ Continuous feedback and updates

→ Advantages

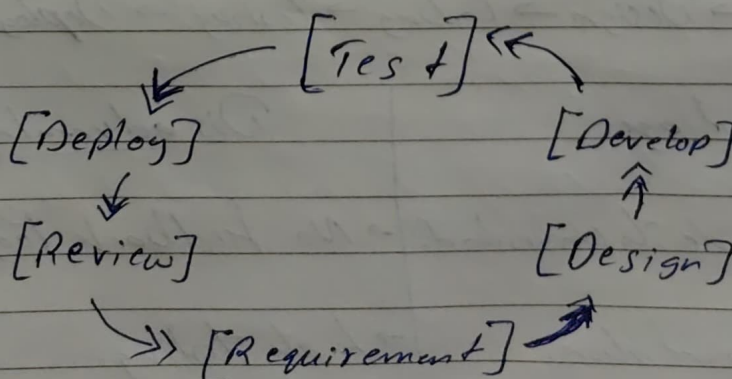
↳ Handles changing requirements

↳ Frequent working software

→ Disadvantages

↳ Less documentation

↳ Requires skilled team



34 Spiral Model:- This model has characteristics of both iterative and Waterfall models. It is used in project which are large and complex. The no. of loops in the spiral is not decided in advance but it depends on the size of project and the changing requirement of user.

→ Eg:- Banking Software

Each cycle :- Planning → Risk analysis → Prototyping → Testing.

→ Advantages

↳ Excellent for high risk project

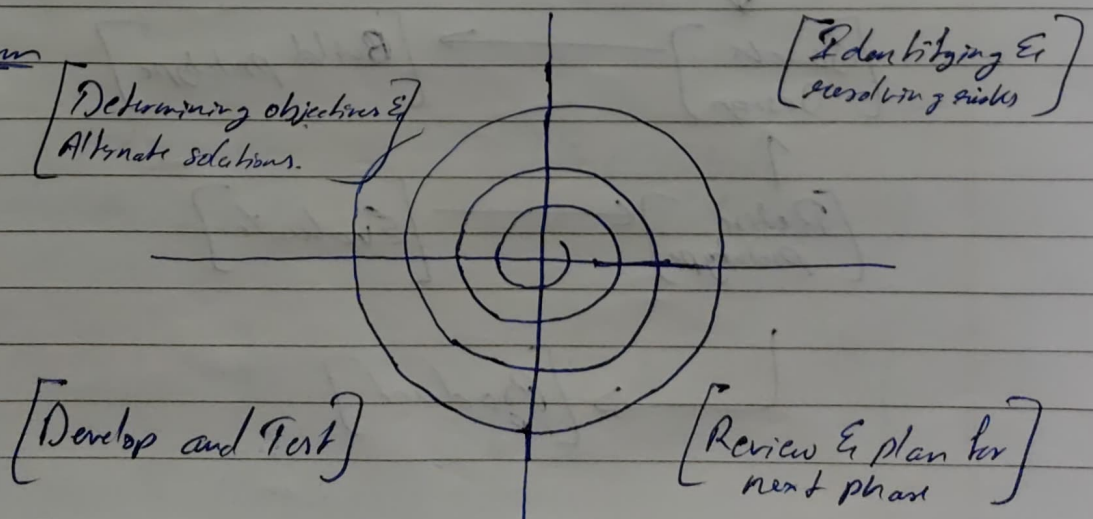
↳ Early detection of risks.

→ Disadvantages

↳ Very expensive

↳ Complex to manage

→ Diagram



4) Prototyping Model :: It is an activity in which prototypes of software are created.

A working prototype is built first so the user gives feedback and then the final system is developed.

→ Eg: Mobile App UI

Build a quick UI prototype → Show to user → get feedback →

→ improve it → final system developed.

→ Advantages

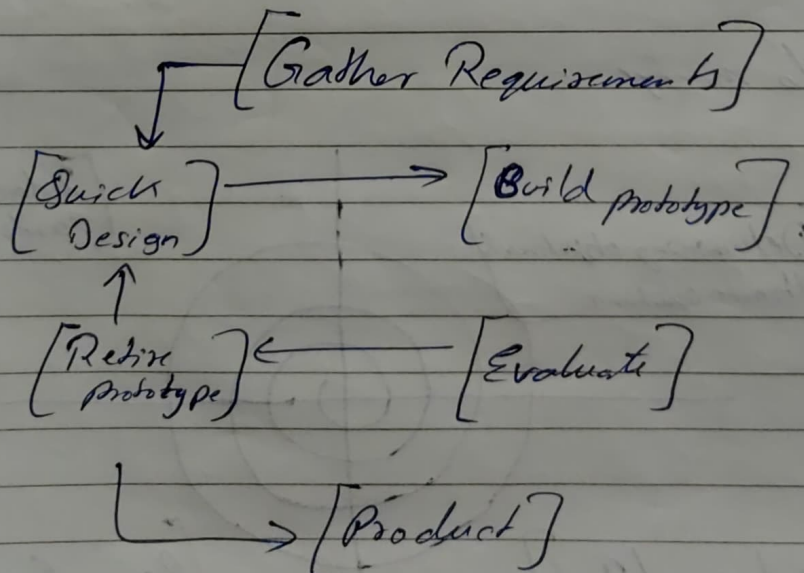
↳ Reduce Misunderstanding

↳ Improve UI design.

→ Disadvantages

↳ Increased cost

↳ Over-dependence on prototype.



57 V-Model :- Also known as verification and validation.

It is used in software development process and it is considered a disciplined model. The execution of each process is sequential, the new phase starts only after the previous phase ~~ends~~ ends.

→ Eg:- Hospital Management system :-

→ Verification on left (SRS → design → coding)

→ Validation on right (UI test → integration test → system test -
→ Acceptance test)

→ Testing happens in parallel with development stages.

→ Advantages

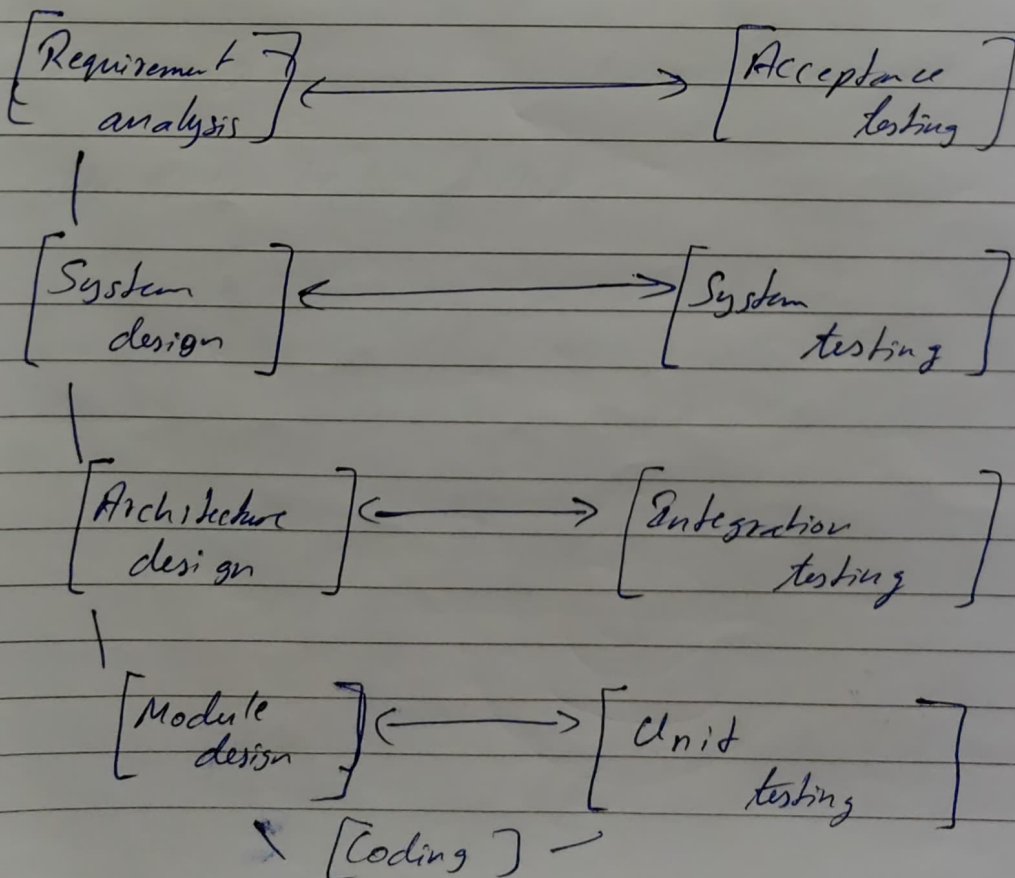
↳ Early test planning.

↳ High quality output.

→ Disadvantages

↳ Rigid.

↳ Costly.



⑤ Role of Software metrics in ensuring project quality.

↳ Software metrics play a critical role in ensuring project quality because they provide objective, quantifiable data about various aspects of software development. By measuring what is happening throughout the project lifecycle, teams can make informed decisions, detect problems early, and deliver higher-quality software.

Highlights :-

1) Improve Planning and Estimation

↳ Metrics such as effort estimation, velocity, function points, and story points help -

→ Predict project timelines and resource needs.

→ Plan iterations more realistically

→ Reduce risks caused by under- or -over estimation.

2) Enable Early detection of Defects

↳ Quality-related metrics - like defect density, defect leakage, and mean time to detect (MTTD) - allows teams to -

→ Identify areas of the codebase that are error-prone.

→ Catch problems early before they become costly.

→ Measure the effectiveness of testing activities.

3) Monitor Code Quality

↳ Code metrics such as cyclomatic complexity, code churn, maintainability index, and test coverage help ensure :-

- Code is modular and easy to maintain
- Complexity is controlled
- Adequate testing prevents regression issues.

These metrics make it easier to enforce coding standards and maintain long-term software health.

4) Enhance process improvement

↳ Process metrics allow teams to :-

- Analyze bottlenecks in development or testing
- Improve workflows and productivity.
- Implement continuous improvement (Kaizen)

5) Increase Transparency and accountability

↳ Metrics create visibility across the project by showing:-

- Progress towards milestones
- Quality levels at each stage.
- How individual teams or companies are performing.

This fosters better communication with stakeholders and enables fact-based decisions.

6) Support Risk Management

- 7) Validate whether requirements are being met.